Exercício Prático 4 – Arq. de Computadores 2

Romanelli

Objetivo: Neste relatório iremos reforçar os conceitos MIPS, MFLOPS, frequência, período e ciclos por instrução (ou CPI).

Introdução:

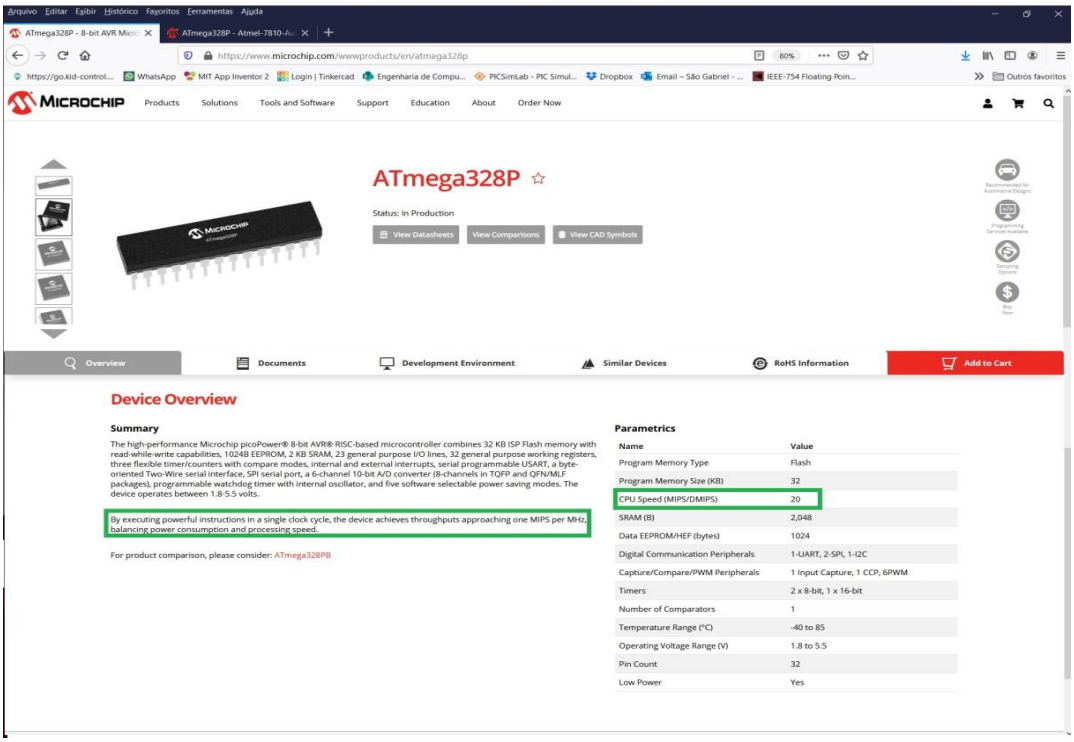
Avaliar um programa medindo o tempo através de um código em alto nível não é uma boa estratégia, incluído neste tempo em geral também estão os tempos necessários do SO, de I/O e também do compilador utilizado.

A melhor estratégia seria construir um programa em uma linguagem de baixo nível (ou a linguagem mais próxima possível do Hardware do processador) e assim, conhecendo-se a quantidade das instruções deste programa, a quantidade de ciclos necessários à execução de cada uma das diferentes instruções deste programa e a frequência de operação do processador, determinaríamos o CPUtime ou o tempo real de execução do código, desconsiderando todas as variáveis incorporadas ao tempo de execução e já mencionadas. Estas são as variáveis que encontramos na Equação de Performance da CPU e que iremos utilizar daqui para frente.

Contudo, nesta experiência não adotaremos esta estratégia acima, nós a iremos realizar durante as aulas teóricas e quando estivermos escrevendo programas em linguagem de baixo nível, mas por agora, iremos construir um método de avaliação e aprender a lidar com os conceitos de MIPS, MFLOPS, quantidade de instruções, ciclos por instrução e frequência.

Experiência Avaliação do Arduino

Se olharmos o datasheet do fabricante do ATM328P, que é o processador da placa do Arduino, podemos ver a velocidade do processador divulgada e que é da ordem de 1 MIPS/MHz.



Como o cristal existente na placa do Arduino é de 16MHz, podemos assumir então uma velocidade de 16 MIPS. 1 MIPS/MHz * 16 MHz = 16 MIPS

Vamos submeter o arduino ao seguinte código:

EP05_teste §

```
long c;
byte i, j;
long inicio, fim, tempo;
void setup() {
Serial.begin(9600);
}

void loop() {
i=1;
j=3;
inicio=micros();
for(c=0;c<1000000;c=c+1) i=j;
fim=micros();
tempo=(fim-inicio);
Serial.print("tempo= ");
Serial.println(tempo);
}
```

A função micros() retorna o tempo em microssegundos desde que o arduino foi iniciado com um erro, para o arduino UNO, de aproximadamente 4 us. Para aqueles que usarão um arduino real, sugiro alguma IDE mais antiga porque nas novas a função micros() não me parece funcionar corretamente.

Assim, se considerarmos que iremos realizar um laço de 1000000 vezes (ou 1M vezes) a variável tempo conterá o tempo que este laço irá demorar. Como está dentro da função loop() iremos ver repetidas vezes este valor. Iremos tomar sempre uma média de uns 10 valores em todas as medidas que iremos realizar.

Vamos realizar os testes em apenas instruções lógicas e aritméticas, mas iremos avaliar os tempos para diferentes tipos (byte, int, float). Assim, também iremos conseguir avaliar, no caso de estarmos com operandos do tipo float, o MFLOAT.

Para criarmos um Baseline, ou uma referencia, deveremos incluir no laço uma atribuição, para evitarmos os erros proporcionados por ela. Resumindo, em todos os tempos das operações medidas iremos subtrair os tempos gastos pelo for e pela atribuição, ficando apenas com os tempos da operação. Lembre-se que estamos com um código em alto nível e existe no meio do processo um compilador que irá traduzir o nosso código para a linguagem de baixo nível do ATM328P.

Para criarmos este Baseline então, usaremos a seguinte linha de código:

for(c=0;c<1000000;c=c+1) i = j ;

Agora você deve preencher o seguinte quadro:

Tipo	Tempo base	Use para o teste (i = i op 3)			Use para o teste (i = i op j)		
		Soma	*Or*	*Mult*	*Soma*	*Or*	*Mult*
byte							
int							
float			XXXX			XXXX	

Uma vez de posse dos valores, calcule a quantidade de MIPS de cada instrução para cada tipo de dado. Obviamente para os valores em float estaremos calculando o MFLOAT.
Como estaremos executando 1M instruções (1000000 instruções) a conta irá ficar direta.
Vamos considerar a seguintes leituras (são valores reais medidos em um arduino):

Tipo	Tempo base	Soma	Mult.	Or	Soma	Mult.	Or
byte	754700	1006220	1069104	817580	1006230	1006220	817580

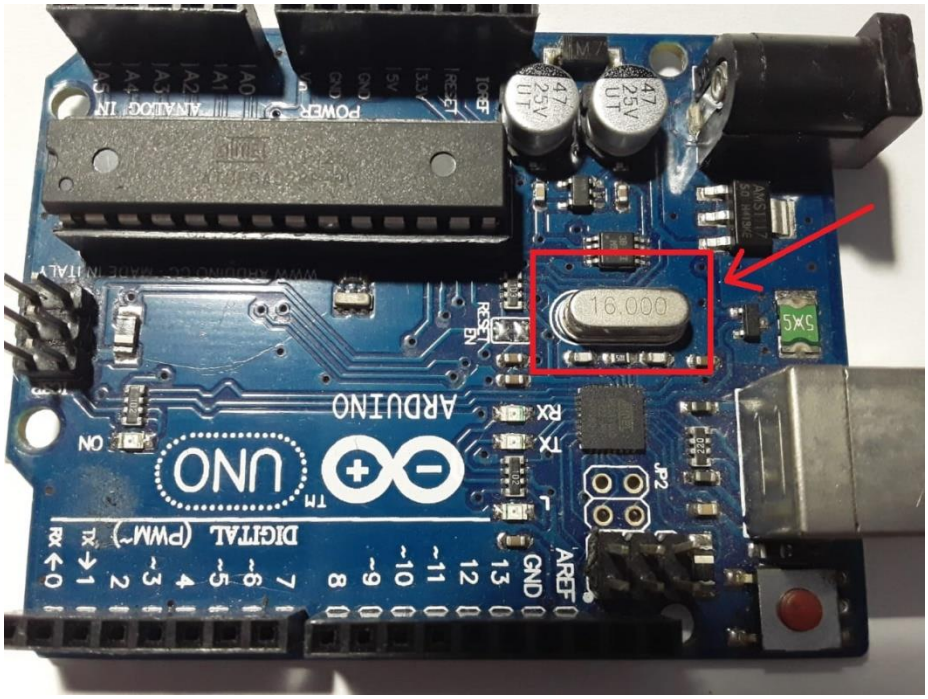
Cálculos:
Tempo de 1000000 somas = 1006220us – 754700us = 251520 us, mas 251520us = 0,251520 s
Assim, realizamos 1000000 somas em 0,251520 segundos ou realizamos 1M somas em 0,251520.
Podemos dizer que estamos com 1 / 0,251520 MIPS = **3,9758 MIPS**.
Com os cálculos realizados preencha a seguinte tabela comparativa:

Tipo	MIPS (ATM328P)					
	Constante (Ex.: i=i op 3 ;)			Variável (Ex.: i=i op j ;)		
	Soma	*Or*	*Mult*	*Soma*	*Or*	*Mult*
byte	**3,9758**					
int						

Tipo	MFLOPS (ATM328P)					
	Constante			Variável		
	Soma	*Or*	*Mult*	*Soma*	*Or*	*Mult*
float		XXXXXXX			XXXXXX	

Podemos ainda tirar algumas conclusões mais interessantes:

O processador ATM328P quando na placa do Arduino está submetido a um cristal que irá fornecer o sinal de clock e que está oscilando a 16 MHz. Veja como identificar o cristal na placa do arduino:



Conhecendo-se então a frequência de clock do processador podemos fazer as seguintes contas:

Frequência de 16MHz = 16×10^6 ciclos/segundos

Mas para o caso do tipo byte e executando uma soma medimos o seguinte:

O processador irá realizar 1×10^6 somas em 0,251520 segundos

Pergunta: Quantos ciclos serão necessários para que ele realize uma soma?

$$\text{Quantidade de ciclos para cada soma} = \frac{(\text{tempo para realizar uma soma})}{(\text{tempo de um ciclo})}$$

Ou

$$\text{Quantidade de ciclos por soma} = (\text{tempo para realizar uma soma}) * \frac{1}{(\text{tempo de um ciclo})}$$

Mas (1 / tempo para realizar um ciclo) é o mesmo que (1/período da onda), ou seja é a frequência da onda.
Com isso podemos reescrever como:
Quantidade de ciclos = tempo para realizar uma soma * frequência da onda

$$\text{Quantidade de ciclos por soma} = \frac{0.251520 \text{ segundos}}{1 \times 10^6 \text{ somas}} * 16 \times 10^6 \text{ ciclos / segundos}$$

Quantidade de ciclos por soma = 0,251520 * 16

Quantidade de ciclos por soma = 4,024 ciclos por soma

O que acabamos de calcular chama-se CPI e quer dizer ciclos por instrução, no caso específico a instrução é uma soma.

Assim, de posse deste número você acabou de formular a equação de performance da CPU:

CPUtime = IC * CPI * 1/f

Onde IC = Quantidade de instruções
CPI = Número de ciclos por cada instrução
1/f = período da onda ou simplesmente 1 / frequência do clock

Vamos verificar se a equação funciona:

IC = quantidade de instruções = 1×10^6 , neste caso estamos tratando das somas realizadas
CPI = 4,024 ciclos por instrução , neste caso a quantidade de ciclos por cada soma que calculamos
1/f = 1 / 16×10^6 Hz, ou a frequência que o processador está submetido

Se efetuarmos a conta teremos:

CPUtime = 1×10^6 * 4,024 * 1 / 16×10^6 = 0,251520

Observe que foi o tempo medido de execução das 1×10^6 instruções ou somas.

Você agora deverá preencher a tabela abaixo com os CPI´s de cada uma das instruções executadas no Arduino:

	CPI					
Tipo	*Soma*	*Or*	*Mult*	*Soma*	*Or*	*Mult*
byte	**4,024**					
Int						
float		XXXX			XXXX	

O que apresentar para este exercício:

- Todas as tabelas preenchidas com as medidas realizadas

- Um printscreen de pelo menos 3 testes no arduino;

- Procure avaliar outros tipos de instruções, crie um pequeno programa de teste e calcule o tempo de execução baseado nestas medições das instruções realizadas. Compare este tempo de execução do seu programa de teste com o tempo real medido através da função micros() usada no seu programa de teste. Comente as eventuais diferenças encontradas.